



Instituto Federal do Ceará - Embarcotech

RED – Relatório de Evidência de Desenvolvimento

**Sistema IoT para Medição e Análise de Consumo de
Energia Elétrica**

Autor: José Adriano Filho

Matrícula: 2025101109806

Out/2025

Sumário

1. Resumo da Etapa.....	3
2. Firmware.....	3
2.1 Arquitetura de Software.....	3
2.2 Uso do FreeRTOS	4
3. Hardware.....	5
4. IoT	9
5. Problemas Encontrados e Soluções Adotadas	9
5.1 Problema com o Módulo HLW8032.....	9
5.2 Solução Adotada: Substituição pelo PZEM-004T	10
6. Desenvolvimento de Bibliotecas.....	11
6.1 Sensor AHT10.....	11
6.2 RTC DS3231 com EEPROM.....	11
6.3 Display TFT 1,8” ST7735.....	11
6.4 Sensor PZEM-0004T.....	12
7. Próximos Passos.....	13

1. Resumo da Etapa

Nesta etapa do projeto foi realizado o desenvolvimento e a consolidação da base técnica do sistema de monitoramento de energia elétrica com conectividade IoT. O trabalho envolveu tanto a evolução do **firmware**, quanto o início da **montagem física do hardware**, permitindo testes progressivos e validação incremental dos módulos.

O firmware encontra-se **em desenvolvimento contínuo**, com foco na modularização do código, estabilidade do sistema e integração gradual dos periféricos. Foram desenvolvidas bibliotecas próprias para sensores ambientais, medição de energia, controle de tempo e interface visual, garantindo maior domínio técnico sobre o funcionamento do sistema.

A abordagem adotada nesta fase priorizou confiabilidade, escalabilidade e facilidade de manutenção, preparando o projeto para a etapa de **integração final**, prevista para janeiro de 2026.

2. Firmware

O firmware foi desenvolvido em linguagem C, utilizando o **SDK oficial da Raspberry Pi Pico W**, aliado ao **FreeRTOS** como sistema operacional de tempo real. Essa escolha permitiu a criação de um sistema multitarefa robusto, capaz de executar simultaneamente aquisição de dados, comunicação de rede e gerenciamento de periféricos.

2.1 Arquitetura de Software

A arquitetura do firmware foi projetada de forma **modular**, separando claramente as responsabilidades de cada componente:

- **Aplicação principal (main.c)**
Responsável pela inicialização do sistema, configuração dos periféricos e criação das tasks do FreeRTOS.
- **Gerenciador de Wi-Fi**
Implementa a lógica de conexão e reconexão automática à rede, monitorando o estado da interface e garantindo disponibilidade da comunicação.
- **Gerenciador MQTT**
Responsável pela comunicação com o broker MQTT, utilizando uma arquitetura assíncrona baseada em filas, evitando bloqueios nas tasks de aquisição de dados.
- **Bibliotecas de sensores e periféricos**
Cada sensor e dispositivo possui uma biblioteca dedicada, facilitando manutenção, testes e reutilização do código.

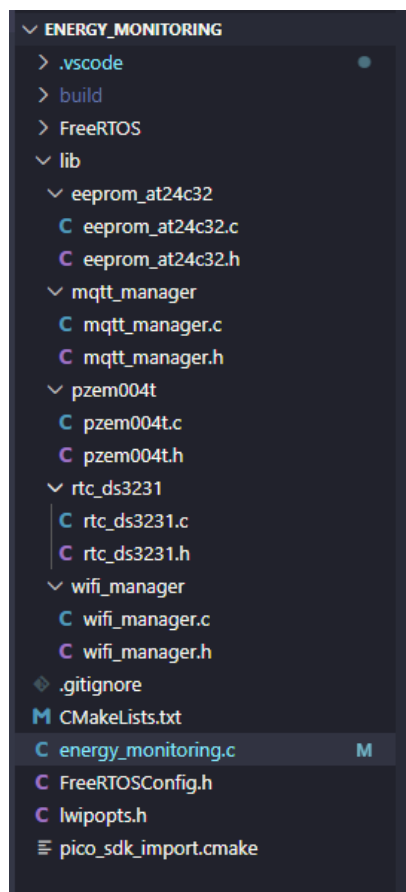


Fig. 1 - Arquitetura inicial do projeto do Firmware

2.2 Uso do FreeRTOS

O FreeRTOS foi empregado para:

- Execução concorrente de múltiplas tarefas;
- Melhor organização do código;
- Separação entre aquisição de dados, processamento e comunicação;
- Maior previsibilidade temporal do sistema.

```
1 #include <stdio.h>
2 #include "pico/stdlib.h"
3 #include "pico/cyw43_arch.h"
4 #include "FreeRTOS.h"
5 #include "task.h"
6
7 #include "wifi_manager.h"
8 #include "mqtt_manager.h"
9 #include "pzem004t.h"
10 #include "rtc_ds3231.h"
11 #include "eeprom_at24c32.h"
12
13 void vTaskSimulatedTemp(void *pvParameters);
14 void vTaskPZEMReader(void *pvParameters);
15 void vTaskRTCReader(void *pvParameters);
16 void vTaskEEPROMTest(void *pvParameters);
```

Fig. 2 – Includes iniciais de bibliotecas

```
// Criar tasks
xTaskCreate(vTaskSimulatedTemp, "TempSimTask", 2048, NULL, 1, NULL);
xTaskCreate(vTaskPZEMReader, "PZEMReader", 4096, NULL, 1, NULL);
xTaskCreate(vTaskRTCReader, "RTCReader", 2048, NULL, 1, NULL);
xTaskCreate(vTaskEEPROMTest, "EEPROMTest", 2048, NULL, 1, NULL);

vTaskStartScheduler();
```

Fig. 3 – Criação das tasks para uso do FreeRTOS

O uso de **tasks**, **filas (queues)** e **semáforos** possibilitou um sistema mais estável e preparado para expansão futura.

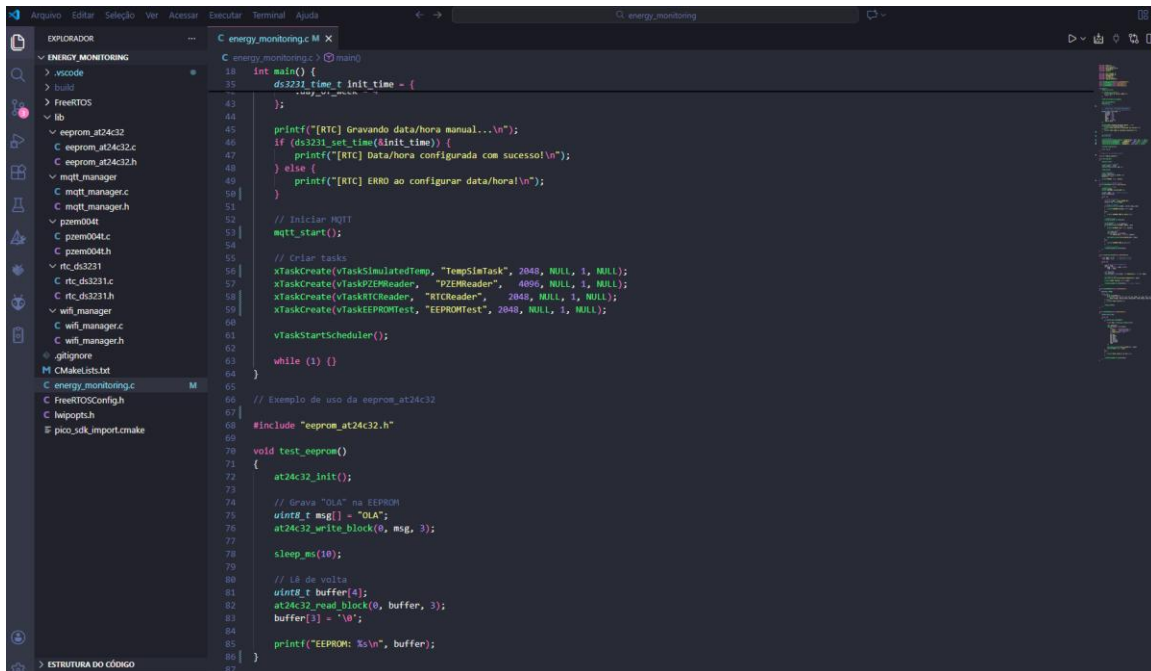


Fig. 4 – Área de trabalho do firmware

Salientamos que o desenvolvimento continua em andamento e que estas estruturas, assim como outras que não estão sendo mostradas poderão sofrer alterações, conforme necessidade durante a implementação e integração final.

3. Hardware

O hardware do sistema foi projetado com foco em modularidade e facilidade de integração. Os principais componentes utilizados são:

- **Microcontrolador:** Raspberry Pi Pico W (RP2040), responsável pelo processamento e conectividade Wi-Fi;
- **Sensor de energia:** PZEM-004T v4.0, conectado via UART;
- **Sensor ambiental:** AHT10, conectado via barramento I²C;

- **RTC:** DS3231 com EEPROM integrada, também via I²C;
- **Display:** TFT de 1,8 polegadas com controlador ST7735, via SPI;
- **Alimentação:** Fonte externa estabilizada.

Nesta etapa, a **montagem física do hardware já foi iniciada**, incluindo a fixação da placa principal e a conexão inicial dos módulos de sensores e comunicação. Os testes estão sendo realizados de forma incremental, permitindo identificar falhas elétricas ou de comunicação antes da integração completa do sistema.

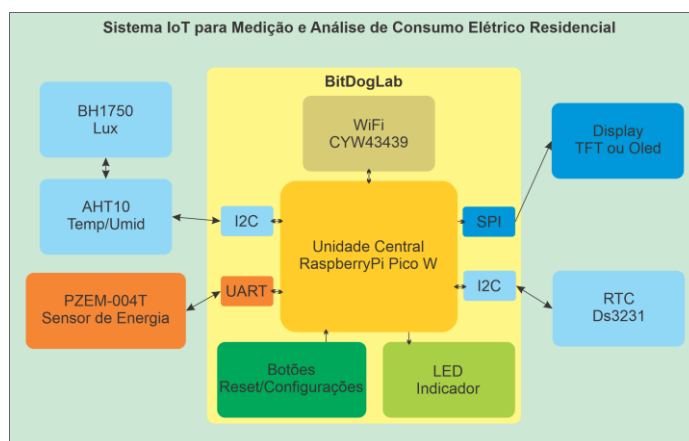


Fig. 5 – Diagrama em blocos – Sujeito a alterações

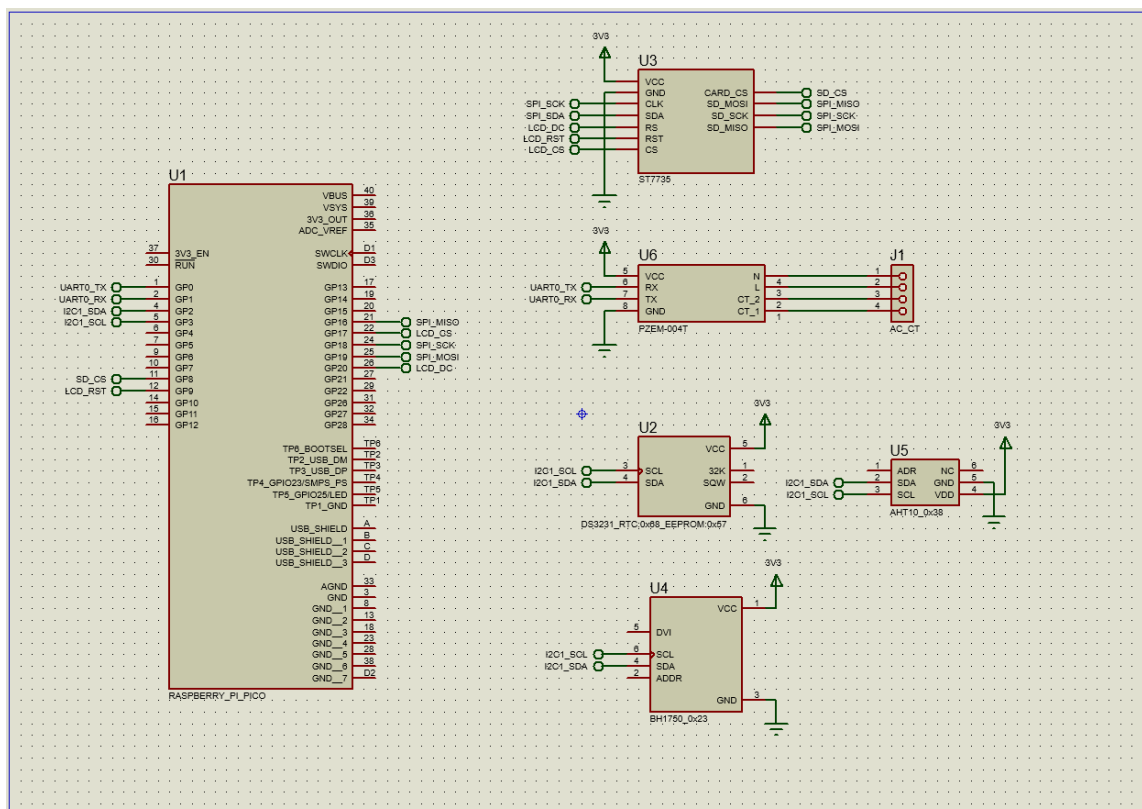


Fig. 6 – Esquemático do projeto – Em construção

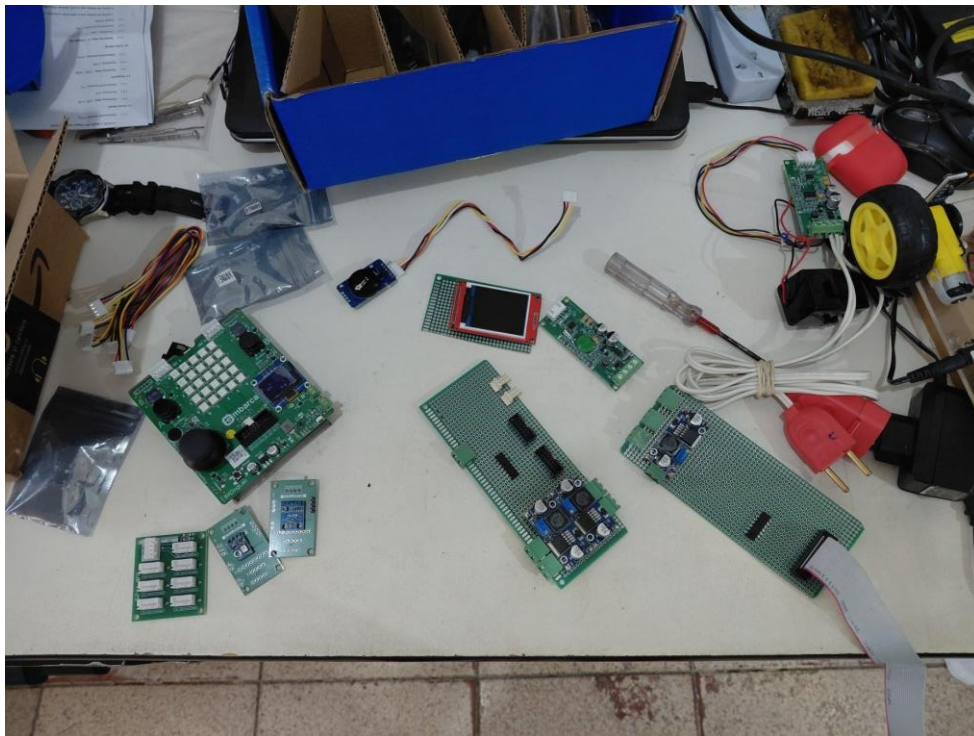


Fig. 7 – Bancada de trabalho



Fig. 8 – Montagem da placa do display

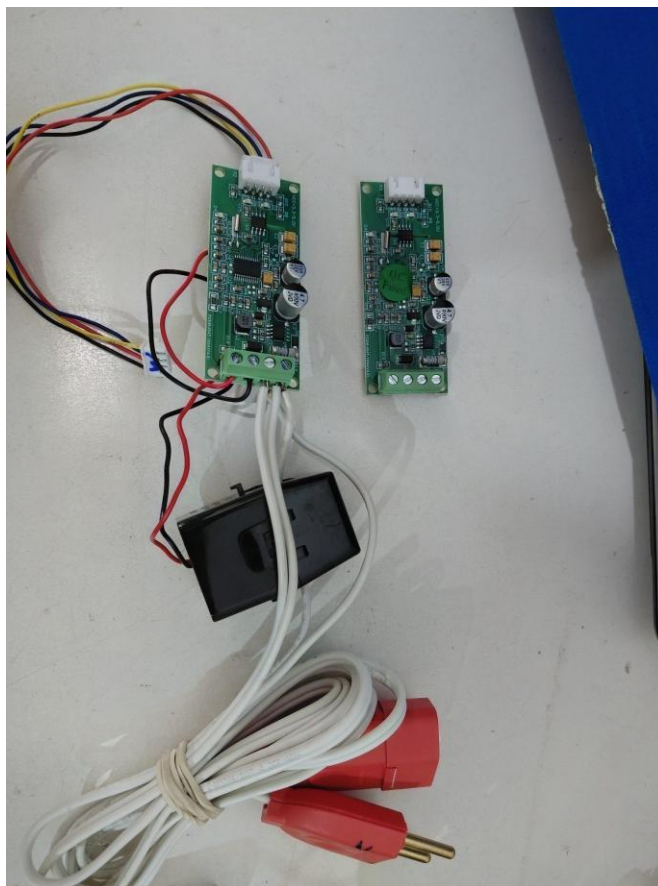


Fig. 9 – Testes com o sensor de aquisição dos dados elétricos

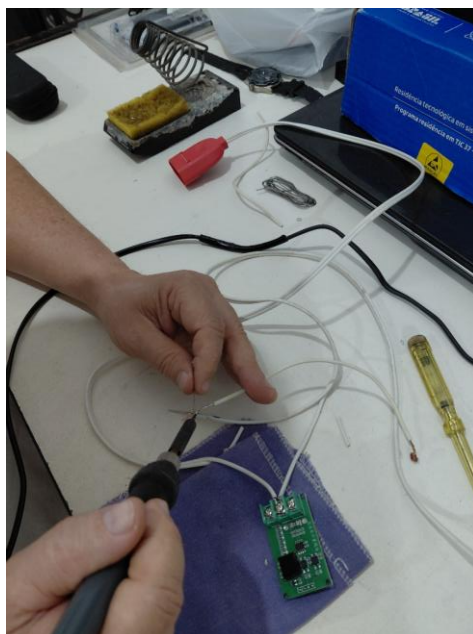


Fig. 10 – Etapas de montagem - soldagem

4. IoT

A camada de IoT do projeto foi implementada utilizando o protocolo **MQTT**, escolhido por sua leveza e ampla adoção em sistemas embarcados.

```
7  #include "wifi_manager.h"
8  #include "mqtt_manager.h"
9  #include "pzem004t.h"
10 #include "rtc_ds3231.h"
11 #include "eeprom_at24c32.h"
```

Fig. 11 – Bibliotecas para WiFi e MQTT

As principais características da implementação incluem:

- Publicação periódica dos dados coletados;
- Estruturação das mensagens em formato **JSON**, facilitando integração com sistemas externos;
- Uso de um broker MQTT para testes e validação;
- Comunicação assíncrona, garantindo que falhas temporárias de rede não interrompam o funcionamento do sistema.

```
123 // Enviar via MQTT
124 char json[128];
125 snprintf(json, sizeof(json),
126          "{\"eeprom_test\": \"%s\"}", buffer);
127
128 mqtt_publish_async("pico/eeprom/test", json);
129 }
```

Fig. 12 – Teste de envio pelo protocolo MQTT

A arquitetura foi projetada para permitir, em etapas futuras, a implementação de **segurança com TLS** e integração com plataformas de visualização e análise de dados.

5. Problemas Encontrados e Soluções Adotadas

5.1 Problema com o Módulo HLW8032

Durante a fase inicial do projeto, foi utilizado um módulo de medição de energia baseado no **HLW8032**. Entretanto, ao longo dos testes, foram identificados diversos problemas, tais como:

- Dificuldade de calibração precisa;
- Leituras inconsistentes em diferentes condições de carga;
- Maior complexidade de implementação no firmware;
- Dificuldade de validação dos resultados obtidos.

Esses fatores comprometeram a confiabilidade do sistema e aumentaram o tempo necessário para desenvolvimento.

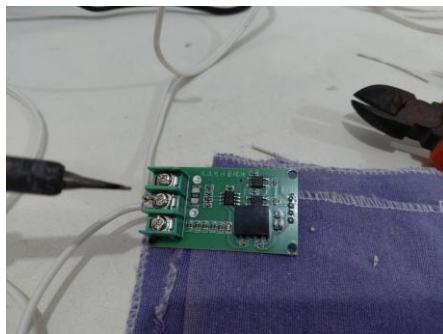


Fig. 13 – Módulo HLW8032

5.2 Solução Adotada: Substituição pelo PZEM-004T

Como solução, optou-se pela substituição do HLW8032 pelo **PZEM-004T v4.0**, que utiliza o protocolo **MODBUS RTU**.

Os benefícios dessa substituição foram:

- Protocolo robusto e bem documentado;
- Leituras estáveis e confiáveis;
- Facilidade de integração com o firmware;
- Redução significativa do tempo de desenvolvimento;
- Maior previsibilidade dos resultados.

Essa decisão contribuiu diretamente para a maturidade e confiabilidade do sistema.

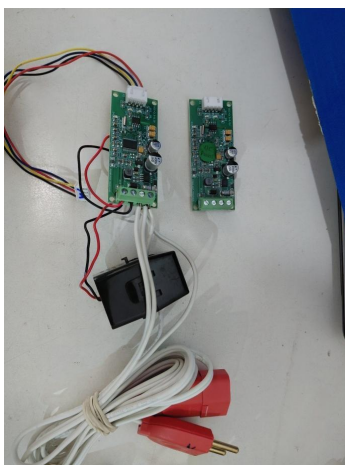


Fig. 14 – Módulo PZEM-0004T

6. Desenvolvimento de Bibliotecas

6.1 Sensor AHT10

Foi desenvolvida uma biblioteca dedicada para o sensor **AHT10**, responsável pela medição de temperatura e umidade.

A biblioteca inclui:

- Rotinas de inicialização do sensor;
- Comunicação via barramento I²C;
- Leitura periódica das grandezas;
- Compatibilidade com execução em ambiente FreeRTOS.

Os testes confirmaram leituras estáveis e confiáveis.

6.2 RTC DS3231 com EEPROM

Também foi desenvolvida uma biblioteca para o **RTC DS3231**, incluindo acesso à **EEPROM** integrada ao módulo.

Funcionalidades implementadas:

- Leitura e escrita de data e hora;
- Conversão entre formatos BCD e decimal;
- Leitura da temperatura interna do RTC;
- Armazenamento persistente de configurações e dados do sistema.

Essa funcionalidade garante manutenção do tempo e dos dados mesmo em caso de desligamento do equipamento.

6.3 Display TFT 1,8” ST7735

Foram realizados testes com um **display TFT de 1,8 polegadas**, baseado no controlador **ST7735**, utilizando comunicação SPI.

Os testes envolveram:

- Inicialização correta do display;
- Escrita de textos e gráficos básicos;
- Exibição de informações do sistema;
- Avaliação do funcionamento em conjunto com o FreeRTOS.

Os resultados indicaram que o display é adequado para uso como interface visual do sistema.

6.4 Sensor PZEM-0004T

Com a utilização do sensor PZEM-0004T para a aquisição de dados sobre o consumo de energia elétrica, e como não existiam bibliotecas para raspberrypi pico W em C, foi necessário desenvolver uma a partir de estudos sobre o sensor.

A biblioteca inclui:

- Rotinas de inicialização do sensor;
- Comunicação via UART;
- Leitura periódica das grandezas;
- Compatibilidade com execução em ambiente FreeRTOS.

Os testes confirmaram leituras estáveis e confiáveis.

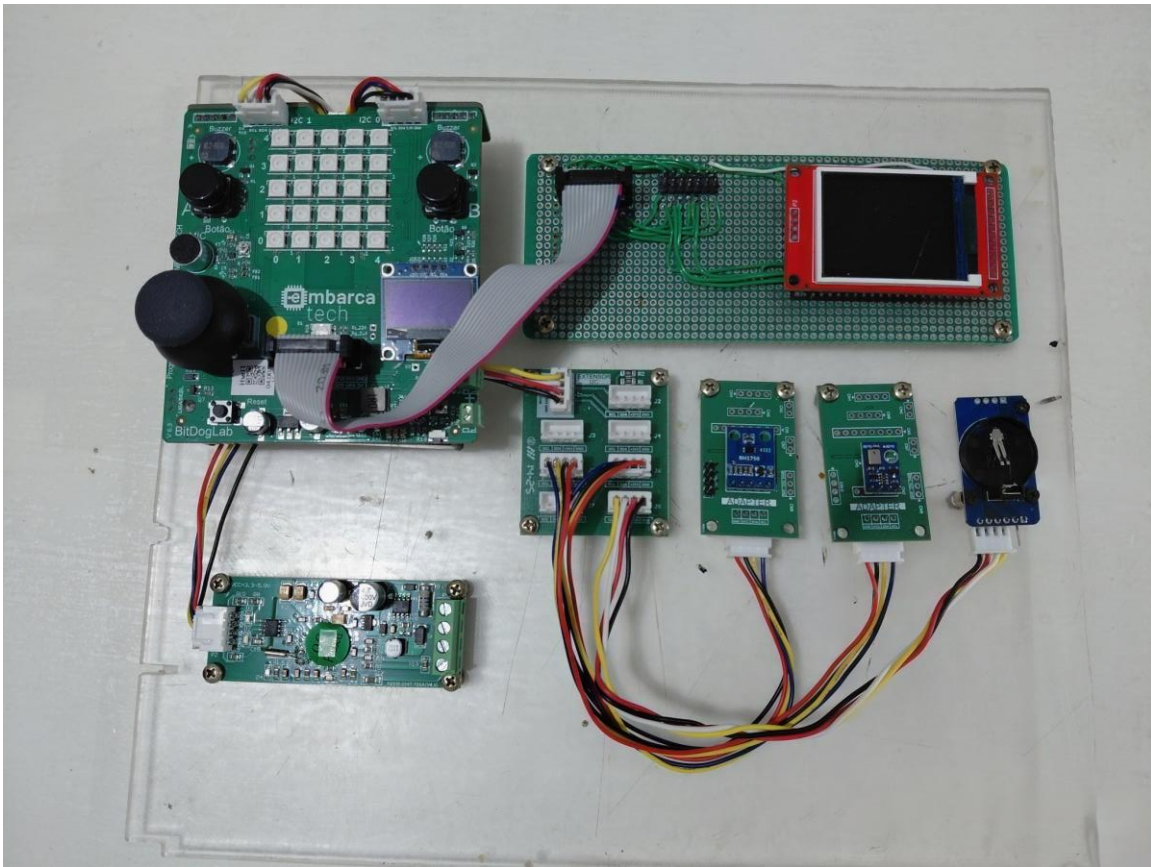


Fig. 15 – Fixação dos módulos para facilitar as conexões e testes

7. Próximos Passos

Os próximos passos do projeto incluem:

- Continuidade do desenvolvimento e refinamento do firmware;
- Finalização da montagem física do hardware;
- Integração completa de todos os módulos;
- Implementação de comunicação MQTT com TLS;
- Uso efetivo do RTC para registro temporal dos dados;
- Armazenamento local em cartão SD;
- Uso do display como interface principal;
- **Integração final do sistema prevista para janeiro de 2026.**